1. Auction Winner Selection Algorithm

```
highest_bid = Bidnow.objects.filter(
    product=auction
).order_by('-bid_amount', 'created_at').first()
```

```
Algorithm Name: Descending Sorting + Maximum Selection Algorithm

Step-by-Step Explanation:
1. Filter all bids for a specific auction product.
2. order_by('-bid_amount'):
   Sorts bids in descending order (highest bid first).
3. 'created_at':
   If two bids have same amount, earliest bid wins (tie-breaking rule).
4. first():
   Selects top element (highest bid after sorting).

Type: Comparison-Based Sorting Algorithm
Time Complexity: O(n log n)
Purpose: Determines auction winner fairly and efficiently.
```

2. Weighted Fraud Risk Scoring Algorithm

```
total_risk = 0
for validation in validations:
    total_risk += validation.get('risk_score', 0)

validation_result['risk_score'] = min(total_risk, 100.0)
```

Algorithm Name: Weighted Risk Scoring Algorithm

Step-by-Step Explanation:
1. Initialize total_risk = 0.
2. Loop through each validation rule result.
3. Add each rule's risk_score to total_risk.
4. Limit final score to maximum of 100 using min().

Mathematical Formula:
Final Risk = Sum(Risk_i)

Type: Aggregation / Weighted Sum Algorithm
Time Complexity: O(n)
Purpose: Combines multiple fraud checks into single decision score.

3. Sliding Window Bid Velocity Algorithm

```
start_time = now - timedelta(seconds=seconds)
recent_bids = Bidnow.objects.filter(
    user=user,
    created_at__gte=start_time
).count()
```

```
Algorithm Name: Sliding Time Window Algorithm

Step-by-Step Explanation:
1. Define time window (e.g., last 60 seconds).
2. Calculate start_time.
3. Count bids made within that window.
4. Compare count against threshold.
5. Increase risk if threshold exceeded.

Type: Sliding Window Counting Algorithm
Time Complexity: O(n) per window
Purpose: Detects rapid automated bidding (bid sniping).
```

4. Sequential Bid Pattern Detection Algorithm

```
differences = [amounts[i] - amounts[i+1] for i in range(len(amounts)-1)]
return len(set(differences)) == 1
```

Algorithm Name: Arithmetic Progression Detection Algorithm

Step-by-Step Explanation:
1. Calculate difference between consecutive bids.
2. Store differences in list.
3. Convert list to set.
4. If set size == 1 → all differences equal → sequential pattern detected.

Mathematical Condition:
a2 - a1 = a3 - a2 = a4 - a3

Type: Pattern Recognition Algorithm
Time Complexity: O(n)
Purpose: Detects bot-like incremental bidding behavior.

5. Running Average Calculation Algorithm

```
profile.average_bid_amount = (
    (profile.average_bid_amount * (profile.total_bids - 1) + bid_amount)
    / profile.total_bids
)
```

```
Algorithm Name: Incremental Mean (Running Average) Algorithm

Step-by-Step Explanation:
1. Multiply old average by (n-1).
2. Add new bid amount.
3. Divide by total bids (n).
4. Store updated average.

Formula:
NewAvg = ((OldAvg × (n-1)) + NewValue) / n

Type: Streaming Data Algorithm
Time Complexity: O(1)
Purpose: Efficiently updates user average without recalculating all bids.
```

6. Threshold-Based Fraud Classification Algorithm

```
if validation_result['risk_score'] >= self.risk_thresholds['high']:
    profile.is_flagged = True
```

Algorithm Name: Threshold-Based Classification Algorithm

Step-by-Step Explanation:
1. Compare computed risk score with predefined threshold.
2. If risk >= threshold → classify as high risk.
3. Flag user or trigger alert.

Type: Decision Boundary Algorithm
Time Complexity: O(1)
Purpose: Converts numeric score into classification result.

7. Machine Learning Fraud Detection (Isolation Forest)

```
self.model = IsolationForest(contamination=0.1)
X_scaled = self.scaler.fit_transform(X)
self.model.fit(X_scaled)

prediction = self.model.predict(X_scaled)[0]
score = self.model.score_samples(X_scaled)[0]

is_fraud = prediction == -1
```

Algorithm Name: Isolation Forest (Unsupervised Anomaly Detection)

Step-by-Step Explanation:
1. Initialize IsolationForest with contamination=0.1 (10% anomalies assumed).
2. Normalize features using StandardScaler.
3. Train model using historical bid data.
4. Predict new bid.
5. prediction == -1 → anomaly detected.
6. score_samples() gives anomaly intensity.

Working Principle:
Fraudulent points are isolated faster in random partition trees.

Type: Machine Learning Anomaly Detection
Time Complexity: O(n log n)
Purpose: Detects unknown and complex fraud patterns automatically.